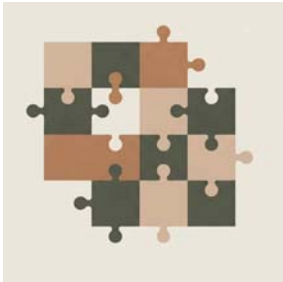


B 3.2.4 Composition and aggregation Python (HL Only)



B3.2.4 Explain the role of composition and aggregation in class relationships. (HL Only)

This section explores composition and aggregation in OOP, focusing on how to design objects using smaller components. It differentiates between aggregation (subcomponents can exist independently) and composition (subcomponents tightly coupled, cannot exist outside the aggregating class).

Composition and Aggregation in Python

Understanding Relationships Between Classes

In Object-Oriented Programming (OOP), classes don't exist in isolation. They often need to interact and work together to model complex systems. Besides inheritance (where one class 'is-a' type of another), two important ways classes relate are through **composition** and **aggregation**. These describe 'has-a' relationships, where one class contains instances of other classes as part of its structure. Understanding the nuances of these relationships is crucial for designing modular and robust software.

Composition: The "Whole-Part" Relationship with Strong Dependency

Composition is a strong type of association where one class (the 'whole') owns and manages the lifecycle of another class (the 'part'). This signifies a strong "has-a" relationship where the 'part' cannot exist independently of the 'whole'. If the 'whole' object is destroyed, the 'part' object also ceases to exist. Think of it like a computer's CPU; it's an integral part of the computer, and if the computer is discarded, the CPU typically goes with it.

Designing Objects with Composition (Python)

To implement composition in Python, you create instances of the 'part' class directly within the `__init__` method of the 'whole' class. The 'whole' class is responsible for creating and managing the 'part' objects.

Example (Python):

```

# Represents the 'part' - a Wheel
class Wheel:
    def __init__(self, material):
        self.material = material
        print(f"Wheel created with {material} material.")

    def get_material(self):
        return self.material

# Represents the 'whole' - a Bicycle
class Bicycle:
    def __init__(self, model, wheel_material):
        self.model = model
        self.front_wheel = Wheel(wheel_material) # Creating Wheel object within B:
        self.back_wheel = Wheel(wheel_material) # Creating another Wheel object
        print(f"{model} bicycle created.")

    def get_model(self):
        return self.model

    def get_front_wheel(self):
        return self.front_wheel

    def get_back_wheel(self):
        return self.back_wheel

# Method to show bicycle details
    def display_details(self):
        print(f"Bicycle Model: {self.model}")
        print(f"Front Wheel Material: {self.front_wheel.get_material()}")
        print(f"Back Wheel Material: {self.back_wheel.get_material()}")

# Example Usage
if __name__ == "__main__":
    my_bicycle = Bicycle("Mountain Bike", "Aluminium")
    my_bicycle.display_details()

    # When my_bicycle object is deleted, the Wheel objects it contains will also be deleted
    del my_bicycle
    print("Bicycle object is now deleted.")
    # Trying to access my_bicycle.front_wheel here would result in an error.

```

In this Python example, the **Bicycle** class **has-a** **front_wheel** and a **back_wheel**, both instances of the **Wheel** class. The **Wheel** objects are created *within* the **__init__** method of the **Bicycle** class.

Their existence is tightly coupled with the `Bicycle` object. When the `my_bicycle` object is deleted using `del`, the `Wheel` objects it contained are also deleted, demonstrating the strong dependency in composition.

Aggregation: The "Has-a" Relationship with Independent Lifecycles

Aggregation is a weaker form of association that also represents a "has-a" relationship. However, in aggregation, the 'part' object can exist independently of the 'whole' object. The 'whole' object contains or uses the 'part' object, but it doesn't necessarily own its entire lifecycle. Think of a library and its books. A book can exist even if the library closes down (it might be sold, given away, or moved to another library).

Designing Objects with Aggregation (Python)

In Python, aggregation is typically implemented by passing instances of the 'part' class to the 'whole' class, often through the `__init__` method or other methods. The 'whole' class uses the 'part' object but doesn't create or destroy it.

Example (Python):

```

# Represents the 'part' - a Book
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        print(f"Book '{title}' by {author} created.")

    def get_title(self):
        return self.title

    def get_author(self):
        return self.author

# Represents the 'whole' - a Library
class Library:
    def __init__(self, name):
        self.name = name
        self.books = []
        print(f"Library '{name}' created.")

    def get_name(self):
        return self.name

    def add_book(self, book):
        self.books.append(book)
        print(f"Book '{book.get_title()}' added to '{self.name}' library.")

    def list_books(self):
        print(f"Books in '{self.name}' library:")
        for book in self.books:
            print(f"- '{book.get_title()}' by {book.get_author()}")

# Example Usage
if __name__ == "__main__":
    book1 = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams")
    book2 = Book("Pride and Prejudice", "Jane Austen")

    city_library = Library("City Central Library")
    city_library.add_book(book1) # Aggregating book1
    city_library.add_book(book2) # Aggregating book2
    city_library.list_books()

# Even if the library object is deleted, the book objects still exist.
del city_library
print("Library object is now deleted.")
print(f"Book '{book1.get_title()}' still exists.")

```

In this Python example, the `Library` class **has-a** relationship with `Book` objects. However, the `Book` objects (`book1` and `book2`) are created *outside* the `Library` class and then passed to it using the `add_book` method. Even when the `city_library` object is deleted, the `book1` object still exists independently, demonstrating the weaker dependency characteristic of aggregation.

Key Differences Summarised (Python)

Feature	Composition	Aggregation
Dependency	Strong; 'part' cannot exist without 'whole'	Weak; 'part' can exist independently of 'whole'
Ownership	'Whole' object owns and manages 'part' object	'Whole' object uses 'part' object but doesn't own it
Creation	'Part' object usually created within 'whole'	'Part' object usually created outside 'whole'
Lifecycle	'Part' object's lifecycle tied to 'whole'	'Part' object's lifecycle independent of 'whole'

Benefits of Composition and Aggregation

- **Modularity:** They allow you to build complex objects from smaller, well-defined components, making your code more organised.
- **Reusability:** Aggregation promotes reusability of 'part' objects as they are not exclusively tied to a single 'whole' object.
- **Maintainability:** Changes within a composed or aggregated object are less likely to have widespread effects on the system, enhancing maintainability.
- **Flexibility:** Choosing between composition and aggregation allows you to model real-world relationships more accurately, leading to more flexible and adaptable designs.

